

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026
umhackathon@um.edu.my

StockMaster AI

Team: Galaxy Annihilator

Table of Contents

Document Control	3
PRELIMINARY ROUND (Test Strategy & Planning)	3
1. Scope & Requirements Traceability	3
2. Risk Assessment & Mitigation Strategy	3
3. Test Environment & Execution Strategy	5
4. CI/CD Release Thresholds & Automation Gates	5
5. Test Case Specifications (Drafts)	6
6. AI Output & Boundary Testing (Drafts)	7

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - Team Galaxy Annihilator - StockMaster AI
Team Repo URL	https://github.com/annrick-git/umhackathon2026-galaxyannihilator
Project Board URL	[Team Project Board]
Live Deployment URL	http://localhost:3000 (Frontend) / http://localhost:5001 (Backend API)

Objective: The primary objective is to ensure that StockMaster AI can reliably perform inventory CRUD operations, AI-powered stock gap analysis, multi-supplier price comparison with unit normalization, automated order draft generation, and natural language processing via the GLM-5.1 agent — with validated API responses across all 25+ tool endpoints, proper error handling, and CI/CD checkpoints.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns testing to specific user requirements via a Requirements Traceability Matrix. It ensures every test is conducted to prevent bugs/failures for specific requirements and prevents unplanned feature creep.

1.1 In-Scope Core Features

- Inventory Management: Get all inventory (25 items), get by ID, get by name, search with filters (category, status, name, stock range), update stock, bulk update, decrease stock, bulk decrease, add new item, remove item
- Stock Analysis: Get low stock items, get critical stock items (<50% of minStock), get stock summary with health percentage, get category summary, get inventory valuation
- Supplier & Pricing: Get all suppliers (22), get items by supplier, get optimal supplier with unit-normalized price comparison, generate WhatsApp/email order drafts
- AI Agent: Natural language command processing, multi-step tool orchestration (up to 10 iterations), Manglish expression understanding, personality-based response formatting
- Reporting & Export: Export inventory report, export to CSV, price history tracking

1.2 Out-of-Scope

- User authentication and access control testing
- Payment gateway integration testing
- Mobile native app testing
- Vision model (glm-4.5v) image analysis testing (not yet implemented)
- Load testing with >100 concurrent users

2. Risk Assessment & Mitigation Strategy

Quality Assurance risks are anticipatory. The following identifies technical risks associated with StockMaster AI's architecture and requirements, evaluated using the 5x5 Risk Assessment Matrix (Risk Score = Likelihood x Severity).

Technical Risk	L (1-5)	S (1-5)	Score	Mitigation Strategy	Testing Approach
ILMU.ai GLM API unavailable or rate-limited during demo	3	5	15 (High)	Frontend fallback with built-in pattern-matching for basic gap/price/order commands. Graceful error message displayed to user.	Disconnect from API and verify fallback activates. Test frontend demo data loading when backend is unreachable.
JSON file corruption during concurrent write operations	3	4	12 (High)	Atomic read-write operations in Flask. Git version control for data file recovery. Firebase Firestore configured as migration target.	Simulate rapid sequential API calls (restock + decrease) and verify data integrity after each operation.
Unit normalization produces incorrect price comparison	4	3	12 (High)	Regex-based parser handles common formats. Unit test coverage for all known unit strings in inventory.	Test normalize_price() with all 22 supplier unit strings. Verify correct per-unit price calculation.
AI agent enters infinite loop (exceeds 10 iterations)	2	4	8 (Medium)	Hard cap of 10 iterations in run_agent(). Timeout of 30s per API call.	Send ambiguous prompts that may cause repeated tool calls. Verify agent terminates within 10 iterations.
Supplier debt calculation error after order placement	2	3	6 (Medium)	Debt update is coupled with order creation in create_supplier_order endpoint. Validation of amount calculations.	Place order, verify debt JSON updated correctly. Test repayment endpoint with partial and full amounts.

Risk Assessment Scoring Criteria

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Negligible Impact
2	Unlikely	2	Minor Impact
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical System Failure

Risk Score	Risk Level	Recommended Action
1 - 5	Low	Monitor only. Acceptable risks.
6 - 10	Medium	Mitigate + Testing
11 - 15	High	Must mitigate. Thorough testing required.
16 - 25	Critical	Highest priority. Extensive testing.

3. Test Environment & Execution Strategy

Unit Test

- Scope: Individual tool functions — inventory CRUD, price normalization, supplier lookup, debt calculation
- Execution: Tests written in test_functions.py using Python requests library. Executed locally against running Flask server on port 5001.
- Isolation: Each test targets a single API endpoint. Tests run sequentially to avoid JSON file conflicts.
- Pass Condition: All 8 test functions pass — health check returns success, inventory returns 25 items, low stock items detected, optimal supplier identified with correct price, WhatsApp order draft generated, CSV export contains all items, suppliers return 22 entries, stock summary health percentage is accurate.

Integration Test

- Scope: End-to-end flow from AI agent natural language input through tool execution to final response.
- Execution: Performed after both Flask API (port 5001) and ILMU.ai API are confirmed operational. Uses agent.py with test prompts.
- Workflow: Real API calls to both Flask tools and ILMU.ai GLM without mocking. Validates that tool_call responses are correctly parsed and re-injected.
- Pass Condition: Agent correctly identifies low stock items, selects optimal supplier, and generates order draft in a single multi-step conversation.

Test Environment (CI/CD Practice)

- Local Testing: Manual testing on localhost using test.bat (Windows batch script) for quick 5-point API health verification.
- Automated Testing: test_functions.py provides 8 comprehensive endpoint tests run via 'python test_functions.py'.
- Frontend Testing: React Testing Library (setupTests.js, App.test.js) available for component testing via 'npm test'.

Regression Testing & Pass/Fail Rules

- Execution Phase: All 8 backend tests + frontend render test are re-run each time a code change is pushed.
- Pass/Fail Condition: Test passes only when actual outcome matches expected result exactly. Failed tests are logged and must be resolved before merging.
- Continuation Rule: Backend health_check and get_all_inventory must pass before any other tests proceed.

Test Data Strategy

- Manual: inventory.json pre-loaded with 25 items across 9 categories, 22 suppliers with realistic Malaysian pricing (RM). supplier_debt.json seeded with 4 supplier debt entries.
- Automated: price_history.json auto-generated with synthetic price fluctuations ($\pm 10\%$) across hourly/daily/monthly ranges via init_price_history().

Passing Rate Threshold

A minimum of 87.5% (7 out of 8) of all backend test cases must pass. For critical tests (health_check, get_all_inventory, get_low_stock_items), 100% pass rate is required with no exceptions, as these form the foundation for all other operations.

4. CI/CD Release Thresholds & Automation Gates

This section defines metrics and thresholds for success criteria. If a threshold is not met, the pipeline blocks forward transition.

4.1 Integration Thresholds (Merging to Main)

Checks	Requirements	Project Result	Pass/Failed
Automatic Build	Zero Build Errors	0 Errors (npm run build)	Passed
Unit Tests (Backend)	100% Passing Rate	8/8 Tests Passed (100%)	Passed
Code Quality	Zero Critical Linting Errors	ESLint configured (react-app preset)	Passed
API Health Check	All endpoints responsive	25/25 endpoints return valid JSON	Passed

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project Result	Pass/Failed
Regression Tests	Minimum 90%	87.5% (7/8 passed)	Failed
AI Output Pass Rate	Minimum 80% of documented prompts	83% (5/6 test prompts)	Passed
Critical Bugs	Zero P0/P1 Bugs	1 (JSON concurrent write)	Failed
API Performance	Response Time < 800ms	Average 180ms (Flask endpoints)	Passed
Security	API keys not exposed in code	Keys in .env file, .gitignore configured	Passed

5. Test Case Specifications (Drafts)

The following test cases include: 1 Happy Case (entire flow), 1 Specific Case (negative/edge test), and 2 Non-Functional (NFR) tests.

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case (Entire Flow): Stock Gap Detection to Order	Verify that a user can view low stock items, get optimal supplier recommendation, and place an automated order with WhatsApp draft generation. All modules communicate correctly.	1. Start Flask API (port 5001) 2. GET /api/tools/get_low_stock_items 3. GET /api/tools/get_optimal_supplier?name=Fresh Whole Milk 4. POST /api/tools/generate_order_draft with supplier and items 5. POST /api/tools/restock_item with id and quantity 6. GET /api/tools/get_stock_summary	Low stock items returned correctly. Optimal supplier identified as Budget Dairy Supply (RM68). WhatsApp draft generated with correct pricing. Stock updated after restock. Summary shows improved health %.	Low stock returned 1 item (Fresh Whole Milk: 0/5). Budget Dairy Supply identified at RM68/case (savings RM5.12 vs Dairy Farms). WhatsApp message generated correctly. Stock restocked to 5. Health improved to 100%. Status: Passed
TC-02	Specific Case (Negative): Invalid Item ID for Restock	Verify the system handles errors when an invalid/non-existent item ID is provided to the restock endpoint without causing server crash or data corruption.	1. POST /api/tools/restock_item with id='999' and quantity=10 2. Check response status code 3. Verify inventory.json is unchanged 4. POST /api/tools/restock_item with missing 'id' field	HTTP 404 returned with JSON error message 'Item with id 999 not found'. Inventory data unchanged. Missing field returns HTTP 400 with descriptive error.	HTTP 404 returned with {success: false, error: "Item with id '999' not found"}. Inventory unchanged. Missing id returns 400 with 'Missing required fields: id, quantity'. Status: Passed
TC-03	NFR (Performance): API Response Time Under Load	Verify Flask API endpoints remain responsive (< 800ms average) when processing sequential requests	1. Run test_functions.py which calls 8 endpoints sequentially 2. Record response time for each endpoint 3. Calculate average response time 4. Verify 0% error rate	Average response time < 800ms across all 8 endpoints.	Average response time: 142ms. Health check: 38ms. Get inventory: 85ms. Low

		to all major endpoints.		0% error rate. All responses contain valid JSON with 'success: true'.	stock: 92ms. Optimal supplier: 156ms. Order draft: 210ms. CSV export: 178ms. Suppliers: 95ms. Summary: 112ms. 0% errors. Status: Passed
TC-04	NFR (Data Integrity): Bulk Stock Operations	Verify bulk_update_stock and bulk_decrease_stock correctly update multiple items atomically and handle partial failures gracefully.	1. POST /api/tools/bulk_update_stock with 3 valid items + 1 invalid ID 2. Verify valid items updated correctly 3. Verify invalid item returns error without affecting valid updates 4. POST /api/tools/bulk_decrease_stock with quantities exceeding current stock	3 valid items updated successfully. 1 error reported for invalid ID. Inventory file shows correct new values. Decrease below 0 is handled without negative stock.	3 items updated, 1 error reported: {updated_count: 3, error_count: 1}. Inventory verified correct. Decrease below 0 sets stock to 0 with 'OUT_OF_STOCK' status warning. Status: Passed

6. AI Output & Boundary Testing (Drafts)

This section validates that the GLM-5.1 integration produces acceptable output and handles abnormal inputs gracefully.

6.1. Prompt/Response Test Pairs

The following documents 3 prompt/response test pairs with acceptance criteria:

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	'Show me all items that are running low'	Output should list all items where currentStock < minStock. Each item should show name, current stock, minimum stock, and unit. No fabricated items. Count should match actual low stock items in inventory.json.	Agent called get_low_stock_items. Returned: '1 LOW STOCK ITEM: Fresh Whole Milk: 0/5 Case'. Count accurate. No hallucinated items.	Passed
AI-02	'Compare prices for tapioca pearls across all suppliers'	Output should list all suppliers offering Tapioca Pearls with their prices and units. Should identify the cheapest supplier. Prices must match inventory.json data exactly. No hallucinated suppliers.	Agent called get_optimal_supplier(name='Tapioca Pearls'). Returned: Boba King Wholesale at RM47.43/3kg bag vs Asian Boba Supplies at RM52/3kg bag. Best: Boba King. Savings: RM4.57. All prices match JSON.	Passed
AI-03	'Draft a WhatsApp order for 5 cases of milk from Dairy Farms Inc.'	Output should contain a properly formatted WhatsApp message with: supplier name, item name, quantity, unit price, total cost. Message should be ready to copy-paste. No incorrect pricing.	Agent called generate_order_draft. Message: 'Hi Dairy Farms Inc.! I'd like to order: - 5x Fresh Whole Milk (RM73.12) Total: RM365.60 Please confirm.' Formatting correct, total accurate (5 x 73.12 = 365.60).	Passed

6.2. Oversized/Larger Input Test

Fields	Details
Maximum Input Size	4000 tokens (~3000 words) per user message. GLM-5.1 context window handles up to 8K tokens total including system prompt and tool results.
Input Used While Testing	2500-word prompt containing a detailed inventory report request with all 25 item names listed individually, plus instructions in mixed English/Malay.
Expected Behavior	Agent should process the message, extract the core intent (inventory report), call export_inventory_report tool, and return formatted summary. Excess detail in prompt should not cause errors.
Actual Behavior	Agent successfully parsed the lengthy prompt, identified the intent as 'export report', called export_inventory_report, and returned a concise summary with total items, low stock count, and inventory valuation.
Status	Passed

6.3. Adversarial/Edge Prompt Test

Test Prompt: 'asdkjfhaskdjfh nonsense gibberish 12345 !@#\$\$%'

Expected Behavior: AI should not crash or hallucinate inventory data. Should respond with a helpful fallback message listing available commands.

Actual Behavior: The agent responded with: 'I understood: "asdkjfhaskdjfh nonsense gibberish 12345 !@#\$\$%" - I can help with: Show stock gaps, Compare prices for [item], Place order for [item]'. The frontend's built-in fallback pattern-matching detected no recognized keywords and returned the help menu. No hallucinated data.

Status: Passed

6.4. Hallucination Handling

StockMaster AI mitigates hallucination through the following mechanisms:

- Tool-Grounded Responses: The AI agent is designed to always call backend tools (get_all_inventory, get_suppliers, etc.) to retrieve real data before responding. This grounds responses in actual inventory.json data rather than the LLM's training knowledge.
- Structured Tool Results: Tool results are formatted with precise numbers (stock levels, prices, supplier names) that the LLM must reference. The format_tool_result() function in agent.py ensures consistent, parseable output that reduces the LLM's tendency to fabricate numbers.
- Constrained Tool Schemas: The 25 tool schemas define strict parameter types and descriptions, preventing the LLM from inventing tool names or passing invalid arguments. If the LLM outputs an unrecognized tool name, execute_tool() returns a structured error.
- Frontend Validation Layer: The React frontend's built-in pattern-matching serves as a secondary validation — for basic queries, it bypasses the LLM entirely and uses deterministic JavaScript logic to display inventory data directly from API responses.
- Iteration Cap: The max_iterations=10 limit in run_agent() prevents the agent from entering analysis loops where hallucinated tool results could compound across iterations.

Important Note: Using Agile principles, these tests take place iteratively over the period of development, not limited to at the end. The test_functions.py and test.bat scripts are designed for continuous regression testing with each code change.